Rate Monotonic

Program:

```
#include <stdio.h>

struct Task {
  int id;
  int execution;
  int period;
  int remaining;
  int next_release;
  int priority;
};

int gcd(int a, int b) {
  while (b != 0) {
    int t = b;
    b = a % b;
    a = t;
  }
  return a;
}

int lcm(int a, int b) { return (a * b) / gcd(a, b); }

int main() {

  printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

  int n, i, t, hyper = 1;

  printf("Enter number of tasks: ");
  scanf("%d", &n);

  struct Task task[n];

  for (i = 0; i < n; i++) {
    printf("\nTask %d Execution Time: ", i + 1);
    scanf("%d", &task[i].execution);

    printf("Task %d Period: ", i + 1);
    scanf("%d", &task[i].period);

    task[i].id = i + 1;
    task[i].remaining = 0;
    task[i].next_release = 0;
  }
```

```
  // Assign priority (smaller period = higher priority)
  for (i = 0; i < n; i++) {
    task[i].priority = task[i].period;
  }

  // Calculate hyperperiod (LCM of all periods)
  for (i = 0; i < n; i++) {
    hyper = lcm(hyper, task[i].period);
  }

  printf("\nHyperperiod = %d\n", hyper);

  printf("\nTime\tRunning Task\n");

  for (t = 0; t < hyper; t++) {
    // Release tasks
    for (i = 0; i < n; i++) {
      if (t == task[i].next_release) {
        task[i].remaining = task[i].execution;
        task[i].next_release += task[i].period;
      }
    }

    int best = -1;

    // Choose highest priority task (smallest period)
    for (i = 0; i < n; i++) {
      if (task[i].remaining > 0) {
        if (best == -1 || task[i].priority < task[best].priority) {
          best = i;
        }
      }
    }

    if (best != -1) {
      printf("%d\tT%d\n", t, task[best].id);
      task[best].remaining--;
    } else {
      printf("%d\tIdle\n", t);
    }
  }

  return 0;
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter number of tasks: 2

Task 1 Execution Time: 1
Task 1 Period: 4

Task 2 Execution Time: 2
Task 2 Period: 5

Hyperperiod = 20

Time    Running Task
0       T1
1       T2
2       T2
3       Idle
4       T1
5       T2
6       T2
7       Idle
8       T1
9       Idle
10      T2
11      T2
12      T1
13      Idle
14      Idle
15      T2
16      T1
17      T2
18      Idle
19      Idle
```

Earliest Deadline First

Program:

```
#include <stdio.h>

struct Task {
  int id;
  int execution;
  int period;
  int deadline; // relative deadline
  int remaining;
  int next_release;
  int abs_deadline; // absolute deadline
};

int gcd(int a, int b) {
  while (b != 0) {
    int t = b;
    b = a % b;
    a = t;
  }
  return a;
}

int lcm(int a, int b) { return (a * b) / gcd(a, b); }

int main() {

  printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

  int n, i, t, hyper = 1;

  printf("Enter number of tasks: ");
  scanf("%d", &n);

  struct Task task[n];

  for (i = 0; i < n; i++) {
    task[i].id = i + 1;

    printf("\nTask %d Execution Time: ", i + 1);
    scanf("%d", &task[i].execution);

    printf("Task %d Period: ", i + 1);
```

```
  scanf("%d", &task[i].period);

  printf("Task %d Deadline: ", i + 1);
  scanf("%d", &task[i].deadline);

  task[i].remaining = 0;
  task[i].next_release = 0;
  task[i].abs_deadline = 0;
}

// Hyperperiod
for (i = 0; i < n; i++) {
  hyper = lcm(hyper, task[i].period);
}

printf("\nHyperperiod = %d\n", hyper);
printf("\nTime\tRunning Task\n");

for (t = 0; t < hyper; t++) {
  // Release tasks
  for (i = 0; i < n; i++) {
    if (t == task[i].next_release) {
      task[i].remaining = task[i].execution;
      task[i].abs_deadline = t + task[i].deadline;
      task[i].next_release += task[i].period;
    }
  }

  int best = -1;

  // Choose earliest deadline
  for (i = 0; i < n; i++) {
    if (task[i].remaining > 0) {
      if (best == -1 || task[i].abs_deadline < task[best].abs_deadline) {
        best = i;
      }
    }
  }

  if (best != -1) {
    printf("%d\tT%d\n", t, task[best].id);
    task[best].remaining--;
  } else {
    printf("%d\tIdle\n", t);
```

```
    }
  }

  return 0;
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter number of tasks: 2

Task 1 Execution Time: 1
Task 1 Period: 4
Task 1 Deadline: 3

Task 2 Execution Time: 2
Task 2 Period: 5
Task 2 Deadline: 5

Hyperperiod = 20

Time    Running Task
0       T1
1       T2
2       T2
3       Idle
4       T1
5       T2
6       T2
7       Idle
8       T1
9       Idle
10      T2
11      T2
12      T1
13      Idle
14      Idle
15      T2
16      T1
17      T2
18      Idle
19      Idle
```